OxyMake: A Formally-Specified, Content-Addressable Workflow Engine

Emmanuel Sérié

Centre de Mathématiques Appliquées (CMAP), CNRS,

École Polytechnique, Institut Polytechnique de Paris,

91128 Palaiseau Cedex, France

June 17, 2026

Abstract

Make-lineage workflow runners decide whether a job must re-run from file-modification times. The timestamp is a proxy for *has this content changed?*, and it breaks under routine operations: a `git checkout`, a tree copy to a new path, or a restore from backup all rewrite mtimes without changing a byte of content. Wherever the raw proxy is trusted—GNU Make, and any engine's pure-mtime fast path—those operations force spurious re-execution of an entire pipeline; and when a timestamp lands the wrong side of a comparison, a stale output is silently reused. Modern runners patch the proxy point-wise: Snakemake 7 records per-output provenance and, as our own benchmark verifies, survives mtime churn unmoved. Content-addressing replaces the proxy instead.

OxyMake[1] is a workflow engine written in Rust and shipped as a single static binary. It replaces the timestamp heuristic with a content-addressable cache key: a BLAKE3 hash of rule source ‖ input content ‖ parameters ‖ environment ‖ platform (Eq. 1). Because the key is a pure function of content, the caching decision survives mtime churn and travels across same-platform machines and shared caches, where timestamps mean nothing. The default validation mode (`mtime+hash`) re-hashes a file whenever its metadata moves—a discipline our own benchmark forced on us, after measuring OxyMake's original pure-mtime default re-running every downstream job on the same checkout that Snakemake's provenance shrugged off (§6.3). Phantom re-runs from mtime perturbation disappear *for declared inputs*—OxyMake has no build sandbox, so an input a rule reads but does not declare is invisible to the key (§3.1)—and deletion cascades transitively to dependents. The workflow itself is a declarative, statically-parseable specification rather than a program, so a tool can read it without executing it. It keeps the Make-lineage rule model—backward-chaining DAG resolution with wildcard-driven genericity—so existing Snakemake pipelines port directly.

The engineering envelope is falsifiable. *DAG resolution* runs in 69 ms at 10 000 jobs against Snakemake 7.32.4's 2.31 s on the same workload (33.3×; measured 2026-06-10 on an Apple M4 Max via the bundled reproducer `bench/snakemake-vs-oxymake/`, Linux/x86_64 re-run pending; §6), with near-linear observed scaling in job count (≈6.6 μs of marginal cost per job)—the 10^5-job point is projected, not yet benchmarked. We are explicit about the trade: on a *cold end-to-end* run OxyMake is 1.25–$2.3\times$ *slower* than Snakemake ($0.80\times$, $0.44\times$, $0.70\times$ at 10^2, 10^3, 10^4 jobs respectively; §6.2). That is the price

[1] Source code: `https://github.com/noogram/oxymake`.

of the content-addressed bookkeeping; OxyMake trades raw end-to-end throughput for content-addressable correctness, and pays it back on the warm re-run that caching exists to serve: 7.54× faster under the metadata fast path, 4.02× under full content re-verification (`-cache-validation=hash`).

Two further properties round out the engine. First, *daemon-free execution with a cooperative multi-session protocol*: each `ox run` is self-contained, with no coordinating daemon; the state layer implements and model-checks the claim/reclaim protocol through which several `ox run` processes will converge on one workspace, with wiring it in as the execution gate staged work—today two concurrent sessions duplicate work safely rather than coordinate. Second, because that path is concurrent state transition across independently-failing peers—structurally the hazard class that led Amazon Web Services to adopt TLA+ at production scale [24, pp. 66–73]—the cross-session safety properties are specified in TLA+ [17] and model-checked, not merely tested. "Model-checked" here means bounded exhaustive checking at 2–4 concurrent sessions, under an assumed-atomic state-database commit; §3.5 states exactly what is verified and what is assumed. An `ox.lock` plan-of-record and an NDJSON event stream let a downstream tool reconstruct exactly what ran.

1 Introduction

Workflow engines in the Make lineage—Snakemake, Nextflow, CWL runners—descend from a change-detection heuristic built on file modification times. mtime is a proxy for the question that actually matters, *has this content changed?*, and it is a leaky one. The timestamp moves whenever the filesystem touches a file, even when the bytes are identical, and it can fail to move when content changes faster than clock resolution or across a path with skewed clocks. A `git checkout`, a tree copy to a new working directory, an NFS mount with drifting time, or a restore from backup all perturb mtimes without touching content. For a user running a multi-day pipeline the consequences cut both ways: a phantom re-run burns compute re-deriving outputs that were already correct, and a *missed* re-run silently ships a stale result into a published artefact. This is the wall a GNU Make user hits the first time a routine `git` operation re-runs an entire campaign—and, as we measured against ourselves, the wall any engine rebuilds for its users the moment it ships a pure-mtime fast path.

Who actually hits the wall: an honest accounting. The wall is real, but it is not where folklore puts it. Our benchmark (§6.3) bumps the mtime of a shared input across the whole harness without changing a byte—what a `git checkout` does—and counts what each engine re-runs. Snakemake 7.32.4 re-runs *zero* jobs: since the 7.x line it records per-output provenance (code, parameters, input set, software environment) instead of comparing live input-versus-output timestamps, and that design shrugs off mtime churn entirely. The engine that re-ran the full downstream radius (two-thirds of the pipeline) was OxyMake's own original default—a pure-mtime fast path added for speed, which reintroduced at the validation layer exactly the failure content-addressing had eliminated at the key layer. We fixed what the measurement indicted: the shipping default is now `mtime+hash`, which re-hashes any file whose metadata moved before deciding. The phantom-re-run claim in this paper is therefore scoped to where it is true: GNU Make's live mtime comparison and any runner's pure-mtime fast path re-run on churn; engines that record provenance, or hash content, do not.

Content-addressing fixes the proxy. The software-deployment literature retired this proxy years ago for package builds. Nix [11] and GNU Guix [7] key build artefacts on a cryptographic hash of all declared inputs rather than on timestamps, so the same inputs always resolve to the same output and a changed input always forces a rebuild. OxyMake brings that discipline up to the orchestration layer: its cache key (Eq. 1) is a BLAKE3 hash of rule source, input content hashes, parameters, environment specification, and platform. The key survives `git checkout`, tree copy, and backup-restore because none of those operations change the content it hashes, and a single edit to one rule re-runs exactly the jobs that edit can affect—no more, no fewer. The guarantee is scoped to *declared* inputs: unlike Nix, OxyMake does not sandbox rule execution, so an undeclared input never enters the key (§3.1). OxyMake validates outputs with a metadata fast path that re-hashes whenever mtime or size change (the `mtime+hash` default), offers a pure mtime check as an opt-in for parity with existing runners, and promotes to full content-addressing via `-cache-validation=hash` when correctness across machines or caches is the priority. Snakemake predates this discipline [16] and remains the dominant runner [22]; since the 7.x line it answers the same hazard differently, with the recorded per-output provenance described above, so that timestamp churn alone never triggers re-execution. That record, however, lives beside one working tree and attests how an output was produced, not what the file now contains: it does not travel across machines or shared caches, and it does not detect an output whose bytes changed underneath it. Nextflow's channel-based runtime [10] and CWL's portable specification [8] retain mtime-era assumptions in their reference implementations.

A fast scheduler, honestly bounded. Correctness is the headline, but it is not bought with speed. OxyMake's backward-chaining resolver materialises the full job graph before execution begins, in time $O(R \times P)$ in the rule count R and the expanded path count P—with all output patterns compiled once up front, so the per-path cost is a single regex match (§6.4). On the resolution phase this is 69 ms at 10 000 jobs against Snakemake 7.32.4's 2.31 s, a 33.3× gap on the same workload (§6). We are deliberate about what that number does and does not claim: it is resolution, not end-to-end, and on a cold end-to-end run OxyMake is *slower* than Snakemake (§6.2). The content-addressed bookkeeping that buys correctness is not free on the cold path; OxyMake trades raw end-to-end throughput for that correctness, and earns it back on the warm re-run—the case the cache exists to serve—at 7.54× on the metadata fast path (4.02× under full content re-verification).

Concurrency that is specified, not just tested. The engine's hardest property is not its speed but its behaviour when several `ox run` processes share one workspace with no coordinating daemon. That path is *concurrent state transition across independently-failing peers*, where safety is a relation between peer states rather than a property of any one peer. The cardinality of reachable interleavings on the multi-session surfaces—claim, reclaim, cancel propagation, post-crash recovery—exceeds any feasible test suite, so a green CI corroborates only an infinitesimal slice of the state space. This is structurally the hazard class that led Amazon Web Services to adopt TLA+ at production scale [24, pp. 66–73], where TLC found seven bugs across ten systems that testing, code review, and fault injection had missed. OxyMake answers it the same way: the cross-session safety invariants are specified in TLA+ [17] and model-checked at bounded scope (§3.5), not left to luck on the test path.

Contributions. We contribute, in order of the systems story above:

1. **Content-addressable caching** with an mtime fast-path, keyed on a BLAKE3 hash of rule source, input content, parameters, environment, and platform—eliminating phantom re-runs for declared inputs and supporting transitive cascade on deletion (§3.1).

2. **Single-binary Rust implementation** with a backward-chaining resolver using precompiled-pattern lookup, delivering sub-second DAG resolution and a 14.9 MB statically-linked executable (§5, §6.4).

3. **Daemon-free execution** with idempotent convergent semantics and a cooperative multi-session claim protocol, implemented in the state layer, whose cross-session safety invariants are specified in TLA+ [17] and model-checked (§4.4, §3.5); wiring the protocol in as the execution gate is staged work.

4. **Three-graph architecture** (RuleGraph → JobGraph → ExecGraph) separating the rule definition, the resolved-and-pruned plan, and the per-run execution trace [13] (§4.1).

5. **A reproducibility lockfile (`ox.lock`)**: a content-hashed plan of record that lets a reader re-derive the same job graph from the same TOML, on the same platform, years later (§6.9).

6. **A machine-readable execution interface** with structured NDJSON event streams and programmatic gate approval, letting a downstream tool consume and audit a run's provenance without scraping terminal output (§5.4).

7. **Polyglot execution spectrum** from shell to in-process `call`-mode with in-memory passing (§4.2).

Running example. To ground the discussion that follows, Listing 1 shows a complete Oxymakefile for a three-stage data pipeline. The file is valid TOML; no embedded Python or custom DSL is needed. Key concepts introduced here—wildcards, named inputs/outputs, `expand`, and the `all` pseudo-rule—are defined precisely in Sections 3–4.

Listing 1: A complete Oxymakefile: generate per-sample data, compute statistics, and merge into a report.

```
ox_version = "0.1"

[config]
samples = ["alpha", "beta", "gamma"]

[rule.all]
input = ["results/report.txt"]

[rule.generate]
output = ["data/{sample}.csv"]
shell = """
echo "word,count" > {output}
for w in the {sample} pipeline workflow; do
    echo "$w,$(( RANDOM % 100 + 1 ))" >> {output}
done
"""
```

```
[rule.stats]
input  = { csv = "data/{sample}.csv" }
output = { txt = "results/{sample}_stats.txt" }
shell  = """
echo "# {sample}: $(tail -n+2 {input.csv} | wc -l) rows" \
     > {output.txt}
"""

[rule.report]
input   = ["results/{sample}_stats.txt"]
output  = ["results/report.txt"]
expand  = "product"
shell   = """
echo "=== Pipeline Report ===" > {output}
cat {input} >> {output}
"""
```

The resolver reads the `all` rule, traces its input `results/report.txt` back through `report` → `stats` → `generate`, expands the `{sample}` wildcard against the three configured values, and produces a nine-job DAG. Content-addressable caching ensures that a re-run after editing only `rule.stats` skips the three `generate` jobs entirely. We refer to this example throughout the paper.

2 Background and Related Work

2.1 File-Based Workflow Systems

The lineage of file-based workflow systems begins with Make [12], which introduced the fundamental abstraction: rules declare outputs, inputs, and commands; the engine constructs a directed acyclic graph (DAG) and executes only the steps whose outputs are missing or out of date. Make's influence is profound—nearly every subsequent workflow tool inherits its backward-chaining resolution model.

Snakemake [16] extended Make with Python-based rule definitions and multi-wildcard pattern matching, making it the dominant workflow system in bioinformatics. Its 2021 update added containerized execution, cloud integration, and module composition [22]. However, Snakemake's Python DSL breaks static analysis and IDE support. Its change detection has evolved from live mtime comparison to recorded per-output provenance in the 7.x line, which survives `git checkout` mtime churn unmoved (verified at every bench scale, §6.3); the record remains bound to a single working tree, however, and does not validate output content, so it neither travels across machines nor detects on-disk corruption.

Nextflow [10] takes a dataflow-oriented approach with channels connecting processes, offering strong containerization support but requiring a Groovy-based DSL. The Common Workflow Language (CWL) [8] provides a platform-independent specification but is verbose and lacks optimization capabilities. Galaxy [2] offers a web-based interface optimized for biologists but sacrifices programmability. The Workflow Description Language (WDL) [35] and its reference engine Cromwell target genomics pipelines with a typed, portable specification, though the execution model is tightly coupled to cloud backends. Pegasus [9] targets large-scale distributed workflows but requires significant infrastructure.

OxyMake inherits Snakemake's core paradigm—backward-chaining DAG resolution with

wildcard-driven genericity—while addressing its implementation limitations through content-addressable caching, a declarative TOML format, and a high-performance Rust engine.

2.2 Build System Theory

Mokhov et al. [20, §3–4] [21, §3] provide the definitive theoretical framework for build systems, decomposing them along two orthogonal axes: the *scheduler*, which decides the order in which tasks run (topological: fixed order from the dependency graph; restarting: re-queues a task when a new dependency is discovered; or suspending: pauses a running task mid-flight to resolve a newly discovered dependency), and the *rebuilder*, which decides whether a task's existing output can be reused (dirty bit: rebuild if any input changed; verifying traces: rebuild only if recorded input/output hashes no longer match; constructive traces: select a previously built output whose inputs match; or deep constructive traces: allow intermediate inputs to differ as long as final content matches). Their key insight is that every build system is a composition of a scheduler and a rebuilder, yielding a two-dimensional design space where existing systems (Make, Shake [19], Bazel [15], Nix) occupy specific cells.

In this taxonomy, OxyMake combines a **topological scheduler** with a **verifying-traces rebuilder** augmented with content-addressing. All dependencies—including wildcard expansions and scatter/gather patterns—are resolved statically by a backward-chaining resolver before the scheduler begins execution. The scheduler then computes a topological order over the fully materialised DAG and dispatches jobs whose upstream dependencies are satisfied, with no runtime dependency discovery or suspend/resume. This positions OxyMake closest to Bazel [15] in scheduling strategy, sharing its static dependency graph and content-addressable caching, while replacing Bazel's Starlark [4] configuration layer with declarative TOML and adding domain-specific features (wildcard expansion, environment management, gates) for scientific workflows. Meta's Buck2 [18] pushes this design further with a fully content-addressed execution model built on the Starlark configuration language; OxyMake shares the content-addressing principle but replaces the Starlark layer with TOML to preserve static parseability.

The distinction between *Applicative* tasks (statically known dependencies) and *Monadic* tasks (dependencies discovered at runtime) clarifies OxyMake's position: its rules are strictly Applicative. Wildcard patterns and scatter/gather configurations *appear* dynamic but are expanded at resolution time—before the job graph is constructed—so the scheduler never needs to suspend a running task to discover new dependencies.

Dolstra's Nix thesis [11, ch. 6] introduced content-addressed storage for software deployment, where store paths encode cryptographic hashes of all build inputs. OxyMake adopts this principle for workflow outputs: the cache key is a BLAKE3 hash of the rule source, input content hashes, parameters, environment specification, and platform. One honest distinction must be drawn: Dolstra's observation that "if a build succeeds, we know that we have specified all the dependencies" rests on Nix's build *sandbox*, which hides undeclared inputs from the builder so that an incomplete declaration fails loudly. OxyMake adopts the content-addressed key but not the sandbox: rules run as ordinary processes, so the key is comprehensive over *declared* inputs only, and input completeness remains the user's obligation (§3.1).

2.3 Reproducibility and FAIR Workflows

This subsection maps the reproducibility property OxyMake delivers onto the existing FAIR workflow literature, and marks the boundary between what the orchestration layer owns and what it delegates to the binary substrate.

The Goble three-layer model. Goble et al. [13] argue that computational workflows are first-class FAIR (Findable, Accessible, Interoperable, Reusable) digital objects, not merely tools for producing FAIR data. They identify three layers—*abstract workflow* (the rule graph, independent of any instance), *concrete workflow* (the resolved DAG bound to specific inputs), and *execution trace* (the per-run provenance record). Each layer requires independent FAIR compliance, and the observation that "a workflow that cannot be readily reused is like a scientific paper that cannot be read" is load-bearing: it is the principle that motivates OxyMake's three-graph architecture (§4.1). The mapping is direct: `RuleGraph` carries the abstract workflow, `JobGraph` (post-resolution, post-cache pruning) carries the concrete workflow, and `ExecGraph` together with the audit-state SQLite tables and the NDJSON event stream carries the execution trace.

The Wilkinson 2025 indicators. Wilkinson et al. [36] operationalise the Goble model with specific indicators for computational workflows. Chue Hong et al. [6] complement this with the FAIR4RS principles for research software, which apply to the OxyMake binary itself as a research artefact. The two papers together define the acceptance criteria a workflow engine must meet to be considered FAIR-native; we report against them in Table 12 and discuss the residuals in §6.9.

The four-layer FAIR reproducibility ladder. Inside the Goble model there is a finer-grained ladder that is useful for placing OxyMake in the existing landscape.

Layer	Witness	Tool examples
L1 – Substrate	Same input hash ⇒ same binary	Guix store, Nix store, Docker digest
L2 – Orchestration	Same plan hash ⇒ same DAG	OxyMake `ox.lock`, Snakemake `report`
L3 – Execution	Same DAG ⇒ same output hashes	OxyMake content-cache, Bazel actions
L4 – Audit	Run trace decoupled from code	OxyMake NDJSON + `state.db`, RO-Crate

Table 1: A four-layer FAIR reproducibility ladder. Each layer provides an independent witness, and the witnesses *compose* rather than overlap. A workflow runner can earn FAIR credit at L2–L4 *without* owning L1, provided it does not silently overwrite the substrate's contract.

Delegating the substrate layer. OxyMake's cache key includes an *environment specification* (e.g., `requirements.txt` content hash, Docker image reference, Guix manifest hash), but treats the substrate's contract as an out-of-model axiom (§3.5, substrate boundary). The engine does not attempt to verify that a `uv.lock` or a Guix manifest is itself bit-reproducible; it records the hash and trusts the substrate to deliver. This delegation is honest: it lets a FAIR-archival reader reproduce the OxyMake-level witnesses (L2–L4) on any substrate that honours the recorded hashes. The guix-cwl reference workflows [29, 37] *illustrate* one such substrate-composition pattern—a CWL workflow run under a Guix-managed environment, with Guix owning L1 and the workflow runner owning L2–L4. Whether OxyMake's orchestration-level contract composes cleanly with such a substrate is a design conjecture and a future direction (§7.3), not a property attested here.

The reproducibility crisis in computational science [27, 32] underscores the need for workflow systems that guarantee deterministic re-execution—and equally underscores the need for honest scope-marking: a runner that absorbs the substrate's contract delivers a false signal

when the substrate changes. OxyMake's content-addressable caching provides a stronger guarantee than timestamp-based systems: same inputs + same rule + same parameters ⇒ same caching decision, on any machine, at any time—but *only at L2–L4*. L1 is delegated, by design, to the substrate of the reader's choice.

2.4 Distributed Execution

DAG-based workflow scheduling on heterogeneous distributed systems is a well-studied problem. The HEFT (Heterogeneous Earliest Finish Time) algorithm [33] and its variants provide efficient heuristics for task placement. Adhikari et al. [1] survey cloud scheduling strategies, identifying the trade-off between makespan minimization and cost optimization.

Apache Airflow [5] popularised DAG-based workflow orchestration for data engineering, modelling pipelines as Python-defined task graphs with pluggable executors. Argo Workflows [3] applies the same DAG model natively on Kubernetes, encoding steps as container specifications in YAML. Both systems excel at scheduling heterogeneous tasks across distributed infrastructure but define workflows imperatively (Python or YAML), limiting static analysis and content-addressable caching.

Frameworks like Ray [23] and Dask [30] provide distributed execution with task-level parallelism but require Python and lack the declarative workflow model of Make-like systems. OxyMake bridges this gap with a scaling ladder: the same declarative workflow runs on a local machine (`-j N`), SLURM cluster (`--executor slurm`), or Kubernetes (`--executor k8s`) without modification.

3 Design Principles

OxyMake's design principles are reverse-engineered from the FAIR-workflow contract laid out in §2.3: each principle exists to defend one or more cells of the four-layer reproducibility ladder (Table 1). The founding principle marks the boundary the engine refuses to cross—it records what the orchestration owns and delegates the rest:

> *The engine records what the orchestration owns (L2–L4); the substrate owns what the substrate owns (L1). Both contracts are written down. Neither absorbs the other.*

A secondary, mechanical principle follows:

> *Rust provides the engine. The workflow provides the intent.*

This establishes a strict separation of concerns: the engine handles DAG resolution, scheduling, caching, and execution mechanics; the workflow definition declares rules, dependencies, and resource requirements. The engine never interprets intent (no heuristics, no hardcoded thresholds), and the workflow never specifies mechanics (no scheduling logic, no cache management).

Goble three-layer mapping. Each of the six following principles is annotated with the Goble layer(s) [13] it defends: A = abstract workflow, C = concrete workflow, T = execution trace. Content-addressable caching (§3.1) defends C+T; daemon-free cooperative execution (§4.4) defends T; the API-equals-CLI principle (§5.4) defends T's machine-readability; the scaling ladder defends A's portability; named invariants and formal specifications (§3.5) provide the *proof of rigour* that makes the T-level audit trail trustworthy; the declarative TOML choice defends A's static parseability.

3.1 Content-Addressable by Default

The source of truth for change detection is *file content*, not timestamps. The cache key for a job is:

$$k = \text{BLAKE3}\big(v \parallel h_{\text{rule}} \parallel h_{\text{inputs}} \parallel h_{\text{params}} \parallel h_{\text{env}} \parallel h_{\text{shell}} \parallel p\big) \tag{1}$$

where v is a key-format version tag (bumping it cleanly invalidates caches written under an older format), h_{rule} is the hash of the rule's source (command, inline code, script reference, or function reference), h_{inputs} is the sorted sequence of $(\text{path}, \text{content hash})$ pairs covering declared inputs, parameter files, and—in script mode—the script file itself, h_{params} captures parameter values, h_{env} captures the environment specification by *content* (e.g., the bytes of the referenced `requirements.txt` or conda YAML, not just its path), h_{shell} is the configured shell executable, and p encodes the platform (OS + architecture). Every component is length-framed with a domain-separation tag, and optional components carry explicit presence tags, so the encoding is injective: two distinct job specifications can never serialize to the same byte stream. Binding each content hash to its path prevents two inputs from exchanging contents without changing the key. Because p is part of the key, cache entries are shared only between machines of the *same* platform; heterogeneous OS/arch cache reuse (e.g., develop on macOS arm64, run on Linux x86_64) never produces a false hit—and never hits at all—and is left as future work. Two residual exclusions remain by design and are documented in §7.2: `call`-mode function bodies (the referenced module is content-tracked only if declared as an input) and mutable container image tags (hashed as written, not resolved to digests).

Cache validation is **pluggable** (ADR-006): users choose between three strategies via `--cache-validation`:

- `mtime+hash` (default) — if mtime or size differ, compute the BLAKE3 hash before declaring a hit or miss. Fast on steady-state, correct on change: same-size corruption with a newer timestamp is detected rather than served.
- `mtime` (opt-in) — pure filesystem metadata (stat calls only). Matches Make/Snakemake behavior; delivers sub-100 ms no-op runs with no dependency on `.oxymake/`. Content is never verified, so this mode is unsuitable for shared or multi-user caches.
- `hash` — always compute BLAKE3 hashes. Required for shared or remote caches and CI reproducibility audits.

The strategy is configurable per invocation (`--cache-validation`), per project (`[config] cache_validation` in Oxymakefile.toml), per environment (`OX_CACHE_VALIDATION`), or globally (`~/.config/oxymake/config.toml`). Remote caches automatically promote to `hash` regardless of the configured strategy.

All dimensions of the cache key must be present from the initial release. Adding a missing dimension later would invalidate the entire cache for all users—an unacceptable cost.

Threat model: undeclared inputs. Equation 1 is sound exactly over what it hashes: *declared* inputs. OxyMake executes rules as ordinary processes, with no sandbox or syscall-level isolation. Anything a rule reads without declaring it—a helper script invoked by the shell command, a binary on `$PATH`, a configuration file, a locale setting, an environment variable outside the declared environment specification—never enters the key, so a change to it produces a *false cache hit*: the engine reuses a stale output while reporting, truthfully by its own lights, that

nothing changed. This is the inverse of the phantom re-run, and it is the more dangerous failure because it is silent. Nix and Guix close this hole with a build sandbox that makes undeclared reads fail at build time; OxyMake deliberately does not (rules must run unmodified user code on hosts where namespace isolation is unavailable or unwanted), so the completeness of the input declaration is trusted, not enforced. The practical mitigations are discipline (declare scripts and tools as inputs; pin the environment via `uv.lock` or an image digest, which folds it into h_{env}) and audit (the `ox.lock` plan of record makes the declared key inputs inspectable). Sandboxed execution as an opt-in enforcement layer is future work (§7.2). Every cache-correctness claim in this paper—including "phantom re-runs disappear"—carries this scope.

3.2 Daemon-Free Cooperative Execution

No daemon process is required. Each `ox run` invocation is self-contained: it resolves the DAG, executes jobs, writes state to `.oxymake/state.db` (SQLite), and exits. For concurrent `ox run` processes, the state layer implements—and model-checks—a cooperative-claim protocol of optimistic-lock SQL transitions specified by `CooperativeClaim.tla` (§3.5); **INV-2** forbids two sessions from claiming the same job and forbids a stale session from holding a lease indefinitely. The specification is written in TLA+ [17]. Wiring this protocol in as the execution gate is staged work: today, two concurrent sessions on overlapping job sets duplicate work safely (jobs are idempotent and state writes are atomic) rather than coordinate. No central orchestrator is needed. An optional `ox serve` mode is available for long-running orchestration but is never required.

3.3 API Equals CLI

The CLI is a thin wrapper over `oxymake-api`, a Rust library crate. Every operation is available both as a Rust function call and as a CLI command with `--json` output. The same invocation serves both humans and downstream tools:

```
# Human-readable
ox run results/all.vcf.gz -j 8

# Machine-readable (same operation)
ox run results/all.vcf.gz -j 8 --json
```

3.4 Scaling Ladder

The same workflow file runs at every scale. The transition from laptop to cluster requires zero workflow changes:

Level	Flag	Mechanism
Sequential	(default)	Single process
Local par.	`-j N`	Tokio thread pool
SLURM	`--exec slurm`	`sbatch`/`sacct`
Kubernetes	`--exec k8s`	K8s Job via kube-rs
Ray	`--exec ray`	Ray Jobs API

All entries assume the `.oxymake/` state directory on local disk; multi-node coordination across NFS, Lustre, or GPFS is future work (§7.2).

3.5 Named Invariants and Formal Specifications

This subsection documents the proof-of-rigour discipline that backs the L4 audit-trail witness of the FAIR ladder (Table 1). It is not the main claim of the paper; it is the discipline that justifies trusting the main claim.

Contingent disposition. This discipline is held as load-bearing under an explicitly *contingent* disposition, decided by pre-mortem #3 (2026-05-29): *Per pre-mortem #3 (2026-05-29), the formal-methods discipline of this project is bound to the drift-tripwire CI (`.github/workflows/drift-tripwire.yml`) as an exogenous referee. If the CI shows 3+ consecutive red builds within any 6-month window, the discipline auto-demotes to "exploratory, not load-bearing" — the named invariants survive as documentation but no longer claim formal-methods rigour. This contingency is binding and operator-irreversible without a public ADR amendment with a second-signatory or commit-trail justification.*

OxyMake's runtime is built from components that are individually deterministic—a worker, a scheduler, an evictor, a cancel path, a state database—yet whose composition is concurrent. A FAIR-archival reader has no way to distinguish "the workflow reproduced" from "the workflow ran once on a single-session, no-fault path that happened to produce the same bytes." The discipline that lets us *claim* reproducibility on the multi-session, fault-tolerant path is formal specification of the cross-session safety properties. Without it, the L4 audit trail is a log of one history out of an unknown number of possible histories.

We call the underlying hazard class **CSTAFP**: *Concurrent State Transitions Across independently-Failing Peers, where safety is a cross-peer relation.* On at least three surfaces (multi-session reclaim, cancel propagation, and post-crash recovery) the state of the system is a *relation* between the states of peers that can each fail independently. The cardinality of reachable interleavings on those surfaces is strictly greater than the cardinality of any feasible test suite; a green CI corroborates an infinitesimal fraction of the state space.

The structural reference is Newcombe et al. [24, pp. 66–73], which reports seven bugs found by TLC in ten AWS systems—bugs that had been missed by testing, code review, static analysis, stress testing, and fault injection. Those systems were orchestrated on deterministic components; the hazard class was concurrent, not adversarial. The structural similarity to `ox run` is what justifies the technique here, not any claim about input non-determinism.

Scope and ratio. OxyMake ships three TLA+ specifications totalling 468 lines (‘.tla‘ source: `CacheConsistency.tla` 139 L, `CooperativeClaim.tla` 161 L, `CancelPropagation.tla` 168 L). Against 58,966 SLOC of Rust workspace code, the formal-specification ratio is **0.79%**, in the same order of magnitude as the ∼1% AWS reports. A fourth spec, `Recovery.tla`, is drafted but not yet committed; it graduates when the `SchedulerState::resume` constructor lands.

Mapping invariants to specifications. Each spec defends one or more named invariants, derived from the codebase's load-bearing properties:

What "model-checked" means here. The title's "formally-specified" is a claim about *bounded model checking*, not proof. TLC exhaustively explores the reachable state space of each spec at small, fixed instance sizes: `CacheConsistency` at 2 workers × 3 rules, `CooperativeClaim` at 3 sessions × 2 jobs (TTL 2, clock bound 4), `CancelPropagation` at 3 jobs. Within those bounds the invariants hold over *every* interleaving—this is the qualitative

Invariant	Property	Spec
OX-1	cache key determinism	`CacheConsistency.tla`
OX-2	no backchannel (info direction)	— (defended by ADRs 001/010)
OX-6	stationary cache safety	`CacheConsistency.tla`
INV-3a	`CancelledNeverCached`	`CancelPropagation.tla`
INV-2	claim atomicity / stale-session reclaim	`CooperativeClaim.tla`
INV-3b	`JobFailedImpliesNoIntent`	`CancelPropagation.tla`
INV-3c	`NoZombieRunning`	`CancelPropagation.tla`
OX-7 (draft)	re-derivability from disk	`Recovery.tla` (pending)

Table 2: Named invariants and the specifications that defend them. OX-1, OX-2, OX-6, OX-7 (draft) name product-level properties; INV-2 and INV-3 name the concurrent safety properties checked by TLC.

step beyond testing, which samples interleavings. Beyond them the invariants are corroborated, not verified; no inductive proof (TLAPS) has been attempted. Two further honesty notes. First, the specs import axioms they do not check (next paragraph); the model-checked guarantee is conditional on those axioms holding in deployment. Second, OX-1 as model-checked is conditional on key purity, and that condition is itself modelled: in `CacheConsistency.tla` each materialisation draws the rule's key from a constant set `KeyVariants`, and a worker crash before registration lets a peer recompute the key for the same rule. Under the shipped configuration (`KeyVariants = {1}` — the key is a pure function of the rule's declared inputs) `CacheKeyDeterminism` holds over all crash/re-claim interleavings; the committed red configuration (`KeyVariants = {1,2}`, modelling an undeclared input leaking into the key) refutes it, which is the evidence the invariant has content rather than being true by construction. Purity of the real BLAKE3 key remains an assumed property, and the genuine completeness risk—undeclared inputs, §3.1—remains out of model.

Substrate boundary. A small spec suite is only as honest as the axioms it imports from its substrate. OxyMake records those axioms in `docs/architecture/boundary.md`—a markdown architecture note naming seven axioms about SQLite, the filesystem, the kernel, and the executor process. Two of them (`StorageDeleteAtomic` and `ExecutorFailureClassification`) were added when `CancelPropagation.tla` brought new substrate dependencies into the proof obligation: widening the interior scope widens the frontier as well, and that frontier is made grep-able rather than implicit. One imported axiom deserves emphasis because it can fail in exactly the deployment where multi-session execution is most tempting: `CooperativeClaim.tla` assumes **`StateDbAtomicCommit`**—that a SQLite `COMMIT` is all-or-nothing with respect to concurrent writers, so the `reclaim_stale_jobs` transaction is atomic. SQLite delivers that on a local filesystem with working POSIX locks; on NFS, Lustre, or GPFS—shared filesystems where one might naturally point several sessions at one workspace—file locking is unreliable and the premise is *false*. This is why OxyMake requires `.oxymake/` on local disk (§7.2): the requirement is what discharges the axiom. Run the state database on NFS and INV-2's model-checked guarantee evaporates with its premise.

Verified vs. assumed. Table 3 draws the line in one place.

Sunset mechanism. The discipline is itself a falsifiable claim. Each spec is reviewed on a six-month cadence (first sunset 2026-12-01) against `spec/tla/REVIEWS.md`. A spec that shows

Verified (TLC, bounded, exhaustive)	Assumed (imported, not checked)
`DoneByClaimHolder`, `AtMostOneClaimer` (INV-2; 3 sessions, 2 jobs; the zombie-terminal-write red configuration refutes the former when the session filter is disabled)	`StateDbAtomicCommit` — SQLite commit atomicity; holds on local disk, *false on NFS/Lustre/GPFS*; discharged by the local-disk requirement (§7.2)
`CacheKeyDeterminism`, `RegisterPrecedesRead`, `NoDoubleRegister` (OX-1/OX-6; 2 workers, 3 rules, worker crashes; the nondeterministic-key red configuration refutes the first)	`ExecutorHonest`, `UserCodeMatchesRule`, `StorageDeleteAtomic`, `ExecutorFailureClassification` (`boundary.md`)
`CancelledNeverCached`, `JobFailedImpliesNoIntent`, `NoZombieRunning` (INV-3; 3 jobs)	BLAKE3 key purity (a pure function of declared inputs — `KeyVariants = {1}` in the model) and input-declaration completeness (§3.1)

Table 3: What the TLA+ suite verifies versus what it assumes. The left column is exhaustively checked by TLC at the stated bounds; the right column is the conditional premise of every left-column guarantee.

zero entries in `spec/tla/TRACES.md` (a TLC-produced trace violating a named invariant) *and* zero entries in `spec/tla/DESIGN-CHANGES.md` (a design change motivated by the spec) within a review window is deleted with a sunset citation. Three consecutive "conditional" reviews mandate deletion. The ledger files thus carry the discipline's own falsifiability: a spec that no one reads is worse than no spec at all, and the sunset catches it. See Appendix A for the full review calendar and ledger pointers.

3.6 Declarative Workflow, Polyglot Execution

The workflow definition is always TOML—declarative, statically parseable, not Turing-complete. This is a deliberate departure from Snakemake's Python DSL. However, individual rules execute in any language through four execution modes forming a spectrum from opaque to optimizable (Section 4.2).

3.7 Errors as First-Class Design

Every error message traces the full causal chain through the DAG: which job failed, what the root cause was (exit code, OOM, missing input), which upstream rule was responsible, and what corrective action is available. In JSON mode, the same structure is machine-parseable, enabling downstream tools to programmatically read error chains and take corrective action.

4 Architecture

4.1 The Three Graphs

OxyMake uses three distinct graph representations at different abstraction levels, following the pattern proven by DataFusion (logical → physical plan), Bazel (target → action graph), and Spark (RDD lineage → stages → tasks).

RuleGraph (logical). The workflow as declared by the user. Nodes are `Rule` objects with unresolved wildcards; edges represent input/output pattern dependencies between rules. The RuleGraph is compact and abstract: one `call` node represents *all* variant-call instances. Operations at this level include cycle detection, rule ambiguity detection, and structural validation. The hierarchical visualization command (`ox dag -group-by stage`) operates on this representation.

JobGraph (physical). The RuleGraph expanded (wildcards resolved, conditional guards evaluated) and then optimized through a series of passes. Nodes are `ConcreteJob` objects with fully resolved wildcards; edges are typed as `Produces`, `Consumes`, or `Blocks`. The JobGraph wraps a `petgraph::DiGraph` [28] with typed node variants (`Job`, `Output`, `Gate`).

Six optimization passes are defined for the JobGraph; cache pruning is fully implemented, while the remaining five are planned:

1. **Cache pruning**: Mark jobs with up-to-date outputs as `Skipped`, using the content-addressable cache (Equation 1).
2. **Task fusion** *(planned)*: Merge sequential `call`-mode jobs into a single process, eliminating intermediate serialization (analogous to Spark stage fusion).
3. **Materialization elimination** *(planned)*: Remove disk writes between consecutive `call`-mode jobs when the executor supports in-memory passing (analogous to DataFusion pipelining).
4. **Group scheduling** *(planned)*: Bundle parallel jobs for batch submission (e.g., one `sbatch` for N SLURM jobs).
5. **Critical path analysis** *(planned)*: Identify the longest dependency chain and prioritize those jobs for earliest dispatch.
6. **Partition planning** *(planned)*: Assign subgraphs to executors based on resource requirements and executor capabilities.

Each pass implements a `trait OptimizationPass` with a single method, enabling composable, testable transformations:

```
optimize(&self, JobGraph) -> Result<(JobGraph, PassResult), Box<dyn Error>>
```

The `ox plan` command exposes each optimization stage, analogous to SQL's `EXPLAIN`.

ExecGraph (runtime). The JobGraph annotated with live execution state. Each node carries a status (Pending → Ready → Running → Completed/Failed/Skipped), runtime metrics (wall time, peak memory), and log paths. The scheduler dispatches `Ready` nodes, the reporter emits events, and crash recovery serializes the ExecGraph to SQLite for resumption.

The full pipeline is: `Oxymakefile.toml` → parse → `RuleGraph` → resolve wildcards → `JobGraph (raw)` → optimization passes → `JobGraph (optimized)` → annotate with runtime state → `ExecGraph` → schedule and execute. Figure 1 illustrates this progression using the demo word-frequency pipeline, and Figure 2 shows the RuleGraph as generated by `ox dag -format dot`.

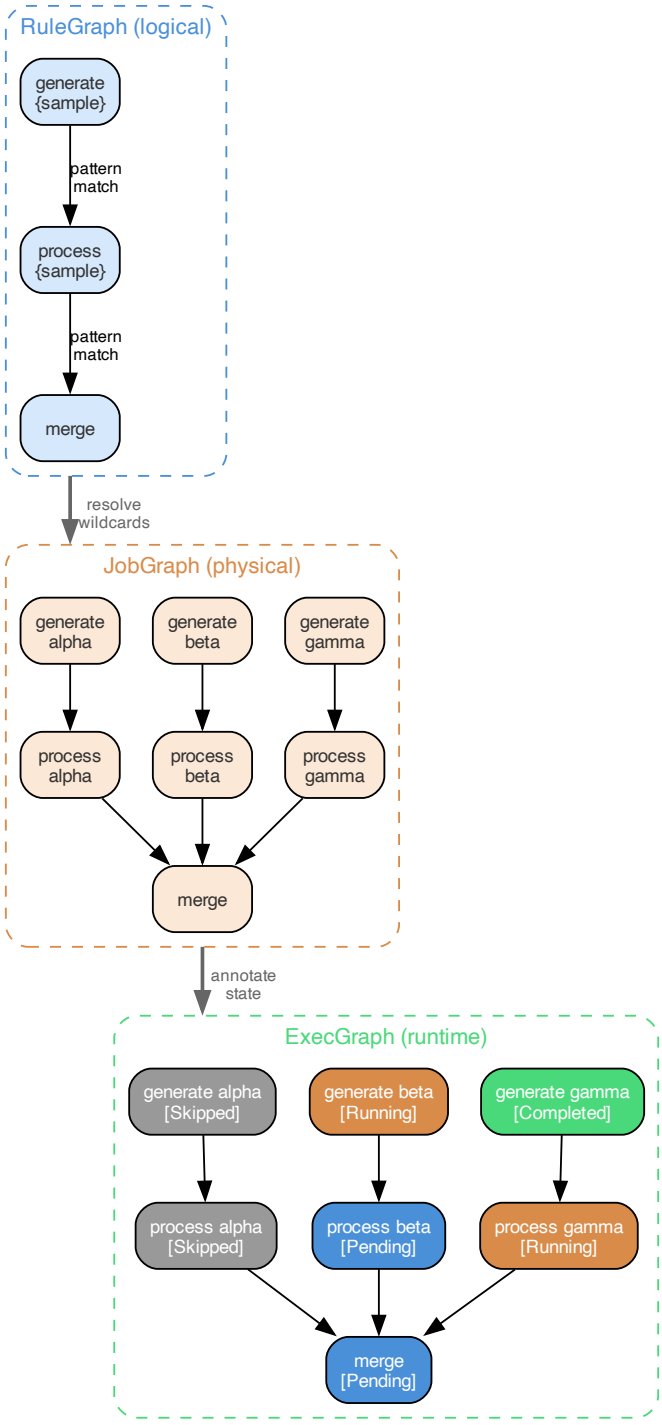

Figure 1: The three-graph architecture applied to the demo pipeline. *RuleGraph* (top): abstract rules with unresolved wildcards. *JobGraph* (middle): wildcards resolved to concrete instances (alpha, beta, gamma), revealing the true parallelism structure. *ExecGraph* (bottom): annotated with runtime state—gray nodes are cache-skipped, orange are running, blue are pending, green are completed.

Figure 2: DAG visualization of the demo word-frequency pipeline generated by `ox dag -format dot`. Rule nodes (blue boxes) are connected through file pattern intermediaries (gray ellipses), forming a bipartite graph that makes data flow explicit. The pipeline progresses left to right: generate → process → merge → target.

4.2 The Execution Spectrum

OxyMake provides four execution modes forming a spectrum from maximum flexibility to maximum optimizability:

Mode	I/O	Memory	Optimize
`shell`	Files only	No	Opaque
`run`	Files only	No	Limited
`script`	Files only	No	Limited
`call`	Files or mem	Yes	Full

The `call` mode is the key innovation. A rule declares a pure function reference (e.g., `pipeline.features:compute_features`); OxyMake manages all I/O outside the function:

```
[rule.compute_features]
input = [{ path = "data/{sample}.parquet",
           format = "parquet" }]
output = [{ path = "features/{sample}.parquet",
            format = "parquet",
            materialize = "auto" }]
call = "pipeline.features:compute_features"
```

The Python function is pure—it receives a DataFrame, returns a DataFrame, and never performs file I/O:

```
def compute_features(df: pl.DataFrame)
    -> pl.DataFrame:
    return df.with_columns(
        anomaly=pl.col("value").rolling_mean(20),
        spread=pl.col("value").rolling_std(60),
    )
```

In file mode, OxyMake reads the input using the declared format codec, calls the function, and writes the result. In memory mode, the transfer mechanism depends on the executor: the Ray executor passes data through the Ray object store (`ray.put`/`ray.get`), avoiding disk entirely, while the local executor currently serializes intermediates to disk via format codecs (Arrow IPC transport is planned). The function never knows the difference.

Materialization policy. The `materialize` field controls when outputs touch disk. Four modes are available: `always` (default, reproducible), `auto` (only if a non-`call` downstream needs the file), `never` (memory only, not cached), and `final` (only DAG leaves). A global

override (`ox run --materialize=final`) switches the entire workflow between modes, enabling rapid exploration with reduced disk overhead (zero on executors with native object stores).

Language interop. OxyMake communicates with language runtimes via subprocess and structured serialization rather than embedding (e.g., PyO3). This design choice (documented in ADR-003) preserves compatibility with isolated environments (`uv`, `conda`, Docker) and avoids coupling to specific language ABIs. Worker processes are reused across sequential `call`-mode jobs in the same environment to amortize startup cost.

4.3 Plugin Architecture

OxyMake defines five plugin axes, each specified by a Rust trait:

Executor Where jobs run: local, SLURM, Kubernetes, Ray.

Storage Where files live: local filesystem, S3, GCS.

Environment How jobs are isolated: system, uv, conda, Docker, Nix.

Reporter How progress is communicated: terminal, NDJSON, webhooks.

FormatCodec How objects are serialized: Parquet, CSV, JSON, and columnar formats.

Plugin selection is compile-time via Cargo feature flags. The minimal binary (local executor + local storage + system environment + terminal reporter + basic codecs) has minimal dependencies and compiles fast. Heavy plugins (Kubernetes, S3) are opt-in features. This ensures the default binary remains small and fast to build.

The `Executor` trait is the most architecturally significant:

```
trait Executor: Send + Sync {
    async fn execute(
        &self, job: &ConcreteJob,
        ws: &Workspace, ctx: &ExecContext
    ) -> Result<JobResult>;
    async fn cancel(&self, id: &JobId)
        -> Result<()>;
    fn capabilities(&self)
        -> ExecutorCapabilities;
}
```

The `capabilities()` method reports whether the executor supports GPU scheduling, streaming between co-scheduled jobs, hermetic workspace isolation, and in-memory passing between `call`-mode jobs. When an executor does not support memory passing (e.g., the local executor's current disk-serialized path, SLURM, Kubernetes), the scheduler automatically promotes `InMemory` outputs to `File`—the workflow runs correctly everywhere, with degraded performance on backends that lack native object stores.

4.4 Idempotent Convergent Execution

`ox run` is not "launch these jobs"—it is "**ensure these outputs exist**." This is a declarative, convergent model inspired by `terraform apply`, formalized as a state-machine reconciliation in the sense of Lamport [17]. Every invocation reconciles desired state versus actual state:

Current State	Action
Output cached, inputs unchanged	Skip
Output missing (intermediate deleted)	Re-execute + cascade
Job running (another session)	Re-execute (safe duplication; attach is staged)
Job pending	Execute
Job failed previously	Re-execute

Transitive invalidation of deleted intermediates. The "output missing" row addresses a subtle correctness property that timestamp-based systems miss. When a user deletes an intermediate output file (e.g., `rm results/merged.parquet`), OxyMake's cache pre-scan detects the absence: the cache store verifies that every recorded output *still exists on disk* before declaring a cache hit. A missing file invalidates the producing job's cache entry and triggers **transitive cascade**—all downstream jobs whose inputs transitively depend on the deleted file are marked stale, regardless of whether their own direct inputs appear intact.

This cascade is implemented by propagating staleness through topological traversal of the job graph: if any upstream job must re-execute, all its direct dependents are marked stale, and this propagation continues until all transitively affected jobs are identified.

Snakemake evaluates each rule against its *direct* inputs and outputs. In some configurations, if an intermediate file is deleted but a downstream output still exists, Snakemake may report "Nothing to be done." Snakemake 7+'s recorded provenance (`--rerun-triggers`) governs re-execution on code, parameter, and input-set changes, but the existence check remains per-rule and local. OxyMake sidesteps the issue entirely: its cache pre-scan verifies that every declared output still exists on disk, so a deleted intermediate always triggers a transitive rebuild. For example, after deleting a single intermediate file in a multi-stage pipeline, `ox run` rebuilds that stage and all downstream stages (preprocess → merge → analyze → report). A minimal reproduction comparing both tools is provided in `examples/intermediate-deletion/`. The property this cascade enforces—no downstream job observes an output whose producer has been invalidated but whose cache entry has not been retired—is defended by the implementation's cache pre-scan and its integration tests; a dedicated specification of the eviction race (`EvictionRace.tla`) is tracked but, like `Recovery.tla` (§3.5), not yet committed. Concurrent sessions on *disjoint* job sets are the supported pattern today:

```
# Terminal 1: dataset-A pipeline
ox run --where dataset=train

# Terminal 2: dataset-B pipeline (concurrent)
ox run --where dataset=test
```

For *overlapping* job sets, the state layer implements a cooperative-claim protocol: atomic SQL updates with optimistic locking (an `UPDATE...WHERE status='pending'` that affects zero rows means another session claimed the job first), session heartbeats, and a two-phase reclaim that resets the jobs of stale sessions (older than 2 minutes) to pending. `CooperativeClaim.tla` model-checks this protocol (§3.5). Wiring it in as the execution gate is staged work: today, overlapping sessions re-execute shared jobs independently—safe duplication (idempotent jobs, atomic state writes), not coordination.

The convergent model yields a closed algebra of workflow control: `ox run` (converge), `ox cancel` (abort), `ox invalidate` (reset), `ox plan` (preview), and `ox status` (observe). All five commands accept `-where`, `-rule`, and `-json` flags.

4.5 Tags, Guards, and Organic Growth

OxyMake provides three mechanisms for managing large, evolving workflows.

Tags. Every resolved wildcard automatically becomes a tag (implicit tags). Rules can also declare explicit tags for cross-cutting concerns (e.g., `stage = "features"`, `cost = "high"`). The `--where` filter selects target jobs by tag values, then backward-chains to include all necessary upstream dependencies. Tags also enable **hierarchical DAG visualization** via `ox dag --group-by stage`, which collapses jobs into meta-nodes by tag grouping—keeping very large DAGs comprehensible at a glance.

Conditional guards. The `when` clause enables non-uniform DAG branches where some wildcard instances need sub-workflows that others do not:

```
[rule.spectral_analysis]
input = ["results/{sample}.parquet"]
output = ["diagnostics/{sample}/outliers.png"]
when = "sample in @high_variance_samples"
```

Guards are evaluated at DAG resolution time—a job whose guard is false is never created in the graph. This enables organic, instance-specific branching without phantom nodes.

Organic growth. Research workflows grow iteratively: explore, select, evaluate, refine. OxyMake supports this through content-addressable incrementality (adding rules never invalidates existing results), workflow composition via `include` directives, snapshots for milestone comparison (`ox snapshot diff baseline-v1`), and run annotations (`ox run –note "Testing new preprocessing step"`) that transform the state database into a lightweight research lab notebook.

5 Implementation

5.1 Rust as Implementation Language

Rust was chosen for three reasons that directly serve OxyMake's design goals.

Type system as architecture. Rust's type system enforces architectural invariants at compile time. `Rule` (unresolved wildcards) and `ConcreteJob` (fully resolved) are distinct types—it is impossible to accidentally schedule an unresolved rule. `Send + Sync` bounds enforce thread safety for the concurrent scheduler. `Result<T, E>` everywhere eliminates the risk of uncaught exceptions killing multi-hour pipelines. The scheduler explicitly cancels all in-flight jobs on shutdown or panic, sending `SIGTERM` to each process group so that child and grandchild processes are cleaned up without orphan leaks.

Performance ceiling. TOML parsing completes in microseconds (versus Python import time of hundreds of milliseconds). The `petgraph` library [28] provides $O(|V| + |E|)$ topological sort via Kahn's algorithm. We have measured resolution up to 50K jobs (Table 6); the near-linear scaling observed and the `ProducerIndex`'s precompiled-pattern resolution (§6.4; asymptotically $O(R \times P)$ with a substantially reduced constant) suggest headroom for 100K-node DAGs, though benchmark validation at that scale is still pending. BLAKE3 [25, 26] hashes

at over 6.9 GiB/s single-threaded on modern x86-64 hardware, exceeding 8 GB/s with AVX-512 SIMD acceleration. The tokio async runtime provides bounded-concurrency scheduling with zero-copy event dispatch.

Distribution. OxyMake ships as a single static binary with no runtime dependencies. Cross-compilation targets Linux, macOS, and Windows. Installation requires only `cargo install oxymake` or downloading a pre-built binary. The 24-crate workspace compiles via standard `cargo build --release`.

5.2 Engineering Methodology

Test-driven development. No code is written without a failing test first. The testing stack includes `cargo test` for unit and doc tests, `cargo-llvm-cov` for line coverage, `proptest` for property-based testing of DAG invariants and cache key stability, `insta` for snapshot testing of TOML parsing and CLI output.

Implementation metrics are reported in Table 5.

Literate programming. Every public function and type has `///` doc comments with executable examples that compile and run as part of `cargo test`. The 55 doc tests serve triple duty: they are documentation, usage examples, and regression tests. Module-level documentation explains design rationale. Architecture Decision Records (ADRs) capture non-obvious choices with context and alternatives. This ensures documentation never drifts from implementation. The project includes 152 documentation files covering concepts, command references, format specifications, and error indices.

Development process. OxyMake was developed iteratively under a specification-driven process: each crate was scoped by its trait boundaries before implementation, and test-driven development was enforced workspace-wide so that every change landed against machine-verifiable acceptance criteria. The full codebase (Table 5) was produced over 19 days across 609 commits.[2]

5.3 Crate Architecture

OxyMake is organized as a Cargo workspace with 24 crates, each with a single, well-defined responsibility. Table 4 summarizes the architectural decomposition, and Table 5 provides concrete implementation metrics collected from the codebase.

The size distribution reveals the expected concentration: `ox-core` contains 29% of the codebase (DAG algorithms, scheduler, type system, event bus) and 39% of unit tests. The four stub crates (`ox-api`, `ox-env-system`, `ox-env-uv`, `ox-storage-local`) contain trait definitions and scaffolding awaiting full implementation. Figure 3 illustrates the dependency relationships between crates.

Each crate has a strict boundary: `ox-core` never performs file I/O or network calls; `ox-format` never validates file existence or executes rules; `ox-state` never decides whether a job should re-run. These boundaries are enforced by Cargo dependency rules—a crate cannot access functionality it does not depend on.

[2]Development used AI-assisted tooling; the methodology and its trade-offs are not a contribution of this paper.

Crate	Responsibility
ox-core	DAG, scheduler, traits, types, events
ox-format	TOML parsing of Oxymakefile
ox-state	SQLite persistence
ox-cache	Content-addressable hashing (BLAKE3)
ox-plan	Optimization passes on JobGraph
ox-api	Public Rust API facade
ox-exec-*	Executor backends (local, SLURM, Ray)
ox-cache-remote	Remote cache backends (S3, GCS, directory)
ox-mcp	Model Context Protocol (MCP) server
ox-storage-*	Storage backends (local, S3)
ox-env-*	Environment providers (system, uv, conda, Docker)
ox-codec-*	Format codecs (CSV, JSON, Parquet, columnar)
ox-report-*	Reporters (terminal, NDJSON)
ox-lock	Reproducibility lockfile (BLAKE3)
ox-translate	Multi-format workflow translator (Snakemake, WDL)
ox-dashboard	Web dashboard server with DAG visualization
ox-monitor-tui	Terminal UI monitor
ox-metrics	Execution metrics collection
ox-cli	CLI binary (clap wrapper on ox-api)

Table 4: Crate responsibilities in the OxyMake workspace.

Crate	Lines	Unit Tests	Doc Tests
ox-core	9,623	322	24
ox-cli	5,354	68	0
ox-translate	4,006	106	1
ox-format	2,521	94	0
ox-state	2,296	27	10
ox-exec-ray	6,320	51	0
ox-exec-slurm	1,991	43	1
ox-exec-local	1,595	44	1
ox-monitor-tui	1,264	23	3
ox-mcp	1,099	0	0
ox-dashboard	922	14	1
ox-lock	609	6	0
ox-cache	554	21	0
ox-report-term	390	17	3
ox-report-json	353	13	1
ox-codec-core	323	20	0
ox-plan	289	10	5
ox-cache-remote	442	5	0
ox-metrics	273	5	3
Other (4 stub crates)	14	0	0
Total	**58,966**	**1,756**	**55**

Table 5: Crate architecture with implementation size and test counts.

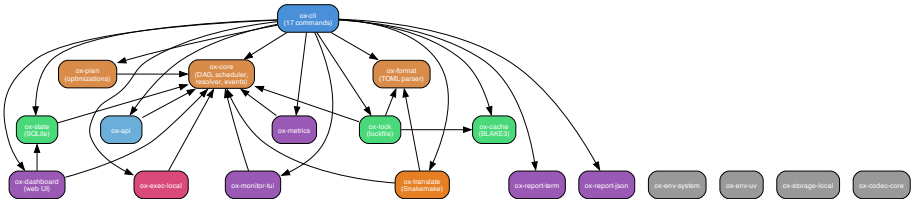

Figure 3: Dependency graph of OxyMake's 24-crate workspace architecture. `ox-cli` sits at the top as the entry point, delegating to `ox-core` (engine), `ox-format` (TOML parser), `ox-state` (SQLite persistence), and specialized crates for execution, reporting, monitoring, and translation.

5.4 Machine-Readable Execution Interface

Every `ox` command supports `-json` for structured output. When active, events are emitted as newline-delimited JSON (NDJSON) on stdout:

```
{"type":"run.started",
 "total_jobs":103429,
 "to_run":847,"cached":102582}
{"type":"job.completed",
 "id":"align_S001","duration_ms":272000}
{"type":"gate.reached",
 "id":"qc_check",
 "message":"Review QC metrics"}
```

A downstream tool reads line-by-line, reacting to typed events. Gate approval is programmatic:

```
ox gate approve qc_check \
  --approver "ci:qc-runner" \
  --reason "metrics within threshold"
```

For Rust embedding, the scheduler exposes a typed `tokio::broadcast` channel of `Event` values, enabling in-process integration without stdout parsing.

5.5 State Management

Three logically separated state concerns share a physical SQLite database (`.oxymake/state.db`):

Execution state (ephemeral): What is running/pending/done. Reconstructible from the DAG and cache if lost. Uses WAL mode and atomic transactions for concurrent access.

Cache state (persistent, immutable): Content-addressable store keyed by Equation 1. Directory structure: `.oxymake/cache/{prefix}/{hash}`. Independent of SQLite—can be shared across same-platform machines via S3.

Audit state (append-only): Run history with notes, per-job metrics (wall time, peak memory, exit code, hostname, environment hash). This provenance record enables reproducibility audits and performance regression detection. It is impossible to backfill if not collected from run 1.

Schema versioning uses `PRAGMA user_version` with automatic migrations from the first release, ensuring no `state.db` is ever left in an inconsistent state.

6 Evaluation

We evaluate OxyMake along seven axes: performance benchmarks on synthetic workflows (Section 6.1), the content-addressing behaviour under mtime churn (Section 6.3), a scalability study with algorithmic optimization (Section 6.4), Snakemake compatibility (Section 6.5), startup overhead (Section 6.6), binary footprint (Section 6.7), and FAIR compliance (Section 6.9). All experiments were run on a single Apple M4 Max (16-core, 128 GB RAM) running Darwin 25.5.0 (arm64); a re-run on a Linux/x86_64 host is pending, and until it lands no cross-architecture claim is made. The DAG-resolution and end-to-end head-to-head numbers (Tables 6 and 7) come from the bundled benchmark of record, `bench/snakemake-vs-oxymake/` (cold wall-clock of each command, median of the configured runs); the startup and binary-footprint micro-benchmarks were timed separately as noted in their subsections.

6.1 DAG Resolution Performance

All numbers in this section come from a **single benchmark of record**: the head-to-head harness shipped at `bench/snakemake-vs-oxymake/`, whose one-line reproducer (`bash bench/snakemake-vs-oxymake/run.sh`) a reviewer can run from the bundle. The harness drives a synthetic four-layer DAG (`seed` $\rightarrow N$ `gen` shell rules $\rightarrow N$ `process` Python-via-shell rules $\rightarrow N$ `finalize` file-copy rules $\rightarrow$ `merge`), totalling $3N{+}2$ jobs; the 10^4 row corresponds to $N{=}3333$ (10,001 jobs). The same DAG and per-job work are declared once as `workflow.toml` (OxyMake) and once as `workflow.smk` (Snakemake 7.32.4); only the orchestrator changes between runs. DAG resolution time was measured using `ox plan` for OxyMake and `snakemake -dryrun` for Snakemake—both parse the workflow, resolve all wildcards, build the job graph, and report the execution plan without running any jobs. The resolution phase is sub-100 ms, so we time it with `hyperfine` (statistical warmup, no subprocess-wrapper overhead); the end-to-end phase below is minutes-scale and timed with `/usr/bin/time`. One scope note applies to every head-to-head number in this section and the next, naming exactly which validation mode produced it. DAG resolution (`ox plan` versus `snakemake -dryrun`) exercises no cache validation at all. The cold and warm end-to-end rows run Snakemake under its default rerun-triggers (recorded per-output provenance) and OxyMake under `mtime` validation (§3.1)—the metadata fast path, which was the shipping default when the record was measured; the default has since moved to `mtime+hash`, which follows the same metadata fast path on an undisturbed tree. A separate warm row re-runs under `–cache-validation=hash`, full content re-verification. The measurements that back the content-addressing thesis are that `hash`-mode row and the `git checkout` scenario—every mtime perturbed, no content changed, count the jobs each engine re-runs—reported in §6.3.
OxyMake resolves a 10^4-job DAG in 69 ms (warm: 27 ms) versus Snakemake's 2.31 s—a 33.3× advantage on DAG resolution. The ratio *narrows* with scale (101.9× at 100 jobs to 33.3× at 10^4) because Snakemake amortises its fixed Python-interpreter startup over more jobs while OxyMake's resolution grows linearly: from 100 to 10^4 jobs (100× more work) OxyMake adds ≈65 ms, a marginal cost of ≈6.6 μs/job, consistent with a per-target lookup whose cost is a handful of precompiled regex matches at this rule count (Section 6.4). **This comparison is scoped to DAG resolution**; the end-to-end head-to-head, where the picture is different, is reported next.

Jobs	OxyMake `ox plan`	Snakemake `--dryrun`	Speedup
100	4 ms	418 ms	101.9×
1,000	10 ms	512 ms	50.7×
10,000	69 ms	2,310 ms	33.3×

Table 6: DAG-resolution wall-clock time, median over hyperfine runs (single benchmark of record, `bench/snakemake-vs-oxymake/`). Measured 2026-06-10, Apple M4 Max (Darwin 25.5.0, 16 cores); OxyMake `ox plan`, Snakemake 7.32.4 `snakemake --dryrun`. Raw measurements and the reproducer in `bench/snakemake-vs-oxymake/RESULTS.md`.

6.2 End-to-End Execution: An Honest Trade

DAG resolution is the phase OxyMake optimises; it is not the whole story. The same harness also measures *end-to-end* wall time—resolve plus execute every job—and here OxyMake is *slower* than Snakemake on a cold run.

Jobs	Snakemake	OxyMake	OxyMake / Snakemake
100	1.10 s	1.37 s	0.80× (slower)
1,000	4.31 s	9.74 s	0.44× (slower)
10,000	1.6 min	2.4 min	0.70× (slower)

Table 7: Cold end-to-end wall time (same benchmark of record). OxyMake runs 1.25–2.3× slower than Snakemake on a cold execution across scales. Warm-cache re-run is the inverse (next paragraph).

We own this result. On a first, cold execution OxyMake pays for what it buys: it hashes every rule's source, inputs, parameters, environment and platform into a BLAKE3 cache key and writes a content-addressed store and an `ox.lock` audit record, where Snakemake checks file mtimes and moves on. That bookkeeping is real wall-clock cost on the cold path. The trade is deliberate—OxyMake exchanges raw cold end-to-end time for content-addressable correctness, cache portability across same-platform machines, and an auditable rebuild decision (§6.9). Three measurements on the *same* harness show where that bookkeeping pays back:

- **Warm-cache re-run** (the common inner-loop case): at 10^4 jobs OxyMake completes a no-op rebuild in 372 ms versus Snakemake's 2.81 s—a 7.54× OxyMake *win*. That row is the `mtime` metadata fast path (the shipping `mtime+hash` default takes the same path on an undisturbed tree); under `--cache-validation=hash`, which re-hashes every input and output before deciding, the same no-op rebuild takes 698 ms—still 4.02× faster than Snakemake, with every byte verified.
- **Minimal-rebuild correctness**: rewriting the content of one Layer-1 input, both systems re-run exactly the 3 affected jobs at every scale—OxyMake's content-addressed decision is as tight as Snakemake's provenance decision, and portable across machines of the same platform where local records are not (the platform term in Eq. 1 scopes the key to one OS/arch pair; heterogeneous reuse is future work).
- **Memory**: peak resident set of the orchestrator on the cold 10^4-job run is 90.7 MiB for OxyMake versus 184.7 MiB for Snakemake—2.04× smaller.

The honest summary: *if your bottleneck is raw cold-run wall time on tiny per-job work, Snakemake is faster today; if it is DAG-resolution latency, warm-cache iteration, same-platform cache reuse, or an auditable rebuild contract, OxyMake is the better engine.* Section 6.4 treats DAG-resolution scaling; the FAIR contract that the cold-path bookkeeping underwrites is evaluated in Section 6.9.

6.3 Content-Addressing under mtime Churn: the git-checkout Test

The scenario that separates timestamp-trusting from content-checking validation is mtime churn: every timestamp moves, no byte changes. The harness reproduces it by bumping the mtime of a shared tracked input (`bench_lib.py`, a declared `lib` input of every `process` job) after a clean build—exactly what a `git checkout`, a tree copy, or a backup-restore does to a working tree. A decision that trusts timestamps must re-run every job that reads the file ($2N+1$: every `process`, every `finalize`, plus `merge`); a decision that checks content must re-run zero.

Jobs	Snakemake 7.32.4	OxyMake `mtime`	OxyMake `hash`
100	0	67	0
1,000	0	667	0
10,000	0	6,667	0

Table 8: Jobs re-run after bumping a shared input's mtime without changing a byte (same benchmark of record, measured 2026-06-10). Validation modes are explicit: Snakemake runs its default rerun-triggers (recorded provenance); the OxyMake columns pin `--cache-validation` to `mtime` and `hash` respectively. The shipping `mtime+hash` default is not a separate measured column; the two measured modes bracket it (see text).

Three findings, stated against ourselves first.

OxyMake's pure-mtime fast path is fooled. It re-runs the full $2N+1$ radius at every scale, because the cheap path trusts exactly the proxy the churn perturbs. This measurement is what flipped the shipping default to `mtime+hash` (§3.1): when a file's metadata moves, the default now re-hashes it before deciding, so a churned tree re-runs zero jobs and pays only the re-hash of the perturbed files—a warm-run cost that falls between the two measured endpoints at 10^4 jobs, 372 ms (metadata only) and 698 ms (re-hash everything).

Snakemake 7.32.4 is not fooled. Its recorded per-output provenance ignores live timestamps: it re-ran zero jobs even with the input's mtime forced to the year 2030, under an explicit `--rerun-triggers mtime`. The phantom-re-run failure mode is real for GNU Make and for any engine's pure-mtime fast path—including, as the middle column shows, our own former default—but the benchmarked Snakemake version does not exhibit it, and we say so plainly.

What content-addressing buys is not this scenario on one machine. On a single working tree, `hash` mode reaches parity with Snakemake's provenance, not superiority. The dividend is everything a working-tree-bound provenance record cannot offer: a caching decision that is a pure function of content, hence valid on any same-platform machine or shared cache (§6.2); detection of on-disk output corruption that a provenance record would serve stale (§3.1); and a rebuild decision auditable from the `ox.lock` record alone (§6.9). It also protects OxyMake's own users from the fast-path footgun the middle column documents.

6.4 Scalability: ProducerIndex Optimization

The measured resolution times in Table 6 ($\leq$69 ms at 10^4 jobs) already meet design targets, but the naive backward-chaining algorithm parses and compiles every candidate output pattern for every target it resolves—$O(R \times P)$ pattern compilations, where R is the number of rules and P the number of output patterns, with a large constant dominated by regex construction. We implemented a `ProducerIndex`: all output patterns are parsed and compiled *once*, up front, so each target lookup is a scan of precompiled regexes rather than a parse-and-compile cycle. The asymptotic complexity remains $O(R \times P)$—lookup is still a linear scan over the compiled patterns—but the constant drops substantially, since per-target regex compilation was the dominant cost at scale. A hash-based prefix index that would reduce lookup to amortized $O(R + P)$ is staged future work, not part of the measured system. The scaling observed across $100 \rightarrow 10^4$ jobs in Table 6 (4 ms $\rightarrow$ 10 ms $\rightarrow$ 69 ms: $\approx$65 ms added for a $100\times$ increase in job count, $\approx$6.6 μs/job) is near-linear in job count because the rule count—and hence the per-target scan cost—stays small and fixed while the job count grows; quadratic behaviour would surface only if rules and targets grew together. Beyond the measured range we report only projections: extrapolating the slope suggests $\sim$0.3 s at 5×10^4 jobs and sub-second at 10^5 jobs. **These are algorithmic-headroom projections, not measurements**—the benchmark of record measures to 10^4 jobs only (5×10^4 and 10^5 are out of scope for this evaluation wave).

6.5 Workflow Language Compatibility

Translation is **bidirectional** and supports multiple source formats. The `ox translate` command converts both Snakemake and WDL workflows to Oxymakefile TOML, while `ox export snakemake` and `ox export wdl` convert an Oxymakefile back to the respective formats. Source format is auto-detected from file extensions (`.smk`, `.wdl`) or content inspection.

Snakemake. We validated Snakemake translation on four real-world workflows of increasing complexity: a minimal two-rule pipeline, a bioinformatics variant-calling workflow with five rules and two wildcards, a multi-sample RNA-seq analysis with conditional rules, and a scatter/gather aggregation pattern. All four translated without manual intervention, and the resulting Oxymakefiles executed with identical output file trees. Translation handles rule definitions, wildcard expansion, `params`/`threads`/`resources` blocks, and `configfile` references. Unsupported Snakemake features (embedded Python expressions, `run:` blocks with arbitrary code) emit structured warnings with migration guidance.

WDL (Workflow Description Language). WDL [35] is the standard workflow language in genomics, used with Cromwell, miniWDL, and the Terra platform. OxyMake's WDL translation maps WDL's `task`/`workflow`/`call` model to OxyMake's rule-based model: each WDL task becomes an OxyMake rule, `command` blocks become `shell` directives, and `runtime` blocks (docker, cpu, memory, disks) map directly to OxyMake's resource and environment declarations. WDL's `scatter` blocks are translated to OxyMake's expand mode, and WDL placeholder syntax (`~{var}` and `${var}`) is converted to OxyMake's `{var}` format. WDL's richer type system (`File`, `Array[File]`, `Int`) produces structured escalations for constructs that require manual review, such as optional inputs (`File?`) and glob expressions.

Why TOML, not WDL? WDL is optimized for a specific domain: bioinformatics pipelines running on cloud infrastructure via engines like Cromwell. Its typed, portable specification

excels at capturing genomics workflows but imposes ceremony that is unnecessary for general computational pipelines: every input must be typed, every task must declare a full runtime block, and the workflow/call/scatter hierarchy adds structural overhead for simple file-to-file transformations. OxyMake's TOML format is domain-agnostic— a three-line rule with `input`, `output`, and `shell` suffices for simple cases, while the same format scales to complex multi-wildcard pipelines with resource declarations. OxyMake is a single statically linked binary with no runtime dependencies; WDL requires a separate execution engine (Cromwell, miniWDL). The bidirectional translator bridges both ecosystems: bioinformatics teams can import their existing WDL workflows into OxyMake for local development and export back to WDL for cloud execution on Terra/Cromwell.

6.6 Startup Time

We measured cold-start time for three commands of increasing complexity:

Command	Wall-clock time
`ox -help`	6 ms
`ox lint` (10 rules)	7 ms
`ox lint` (1,000 rules)	7 ms

Table 9: Startup time (median of 3 runs). Target: $<$200 ms.

Startup time is consistently 6–7 ms regardless of workflow size, confirming that TOML parse time is negligible relative to binary startup. The native Rust binary eliminates Python and JVM startup overhead entirely, achieving 29$\times$ headroom relative to the 200 ms target.

6.7 Binary Size

Metric	Value
Binary size (release, default features)	14.9 MB
Binary type	Mach-O 64-bit arm64
Full release build time	71 s
Target	$<$20 MB

Table 10: Binary footprint and build metrics.

The 14.9 MB binary includes all default-feature crates (24 crates including the TUI monitor, web dashboard, and Snakemake/WDL translator). It is a single statically linked executable with no runtime dependencies, installable via `cargo install` or direct download.

6.8 Implemented vs. Thesis Features

Table 11 provides a transparency assessment of which thesis features are fully implemented, partially implemented, or planned.
Of the 22 features listed, 18 are fully implemented, 1 partial, 2 scaffolded, and 1 planned. The single planned feature (Kubernetes) requires infrastructure that falls outside the core engine and is designated as future work.

Feature	Status	Notes
TOML-based workflow definition	Full	Oxymakefile.toml parsing
Backward-chaining DAG resolution	Full	Wildcards, fan-out
Content-addressable caching	Full	BLAKE3 + mtime fast-path
Three-graph architecture	Full	Rule → Job → Exec
Optimization passes (6 planned)	Partial	Cache pruning implemented
Four execution modes	Full	shell, run, script, call
In-memory data passing	Scaffold	Trait + codec defined
NDJSON event API (`-json`)	Full	All commands
Gate approval	Full	`ox gate approve/reject`
Idempotent convergent execution	Full	SQLite coordination
Tags and `-where` filter	Full	Implicit + explicit tags
Conditional guards (`when`)	Full	DAG-time evaluation
Lockfile (`ox.lock`)	Full	Reproducible plans
Snakemake translator	Full	`ox translate`
WDL translator	Full	`ox translate`, `ox export wdl`
TUI monitor (`ox top`)	Full	Real-time dashboard
Web dashboard	Full	HTML status page
SLURM executor	Full	`-exec slurm`
MCP server	Full	Model Context Protocol endpoint
Kubernetes executor	Planned	`-exec k8s`
Ray executor	Full	`-exec ray`
S3/GCS remote cache	Scaffold	`ox-cache-remote` crate; trait + config (`s3.rs` L19, `gcs.rs` L16 explicitly stubs); HTTP transport not yet wired

Table 11: Feature implementation status. "Full" = tested and functional; "Partial" = core logic implemented, remaining passes planned; "Scaffold" = trait boundaries defined, awaiting implementation; "Planned" = designed but not yet started.

6.9 FAIR Compliance Assessment

We assess OxyMake against the FAIR workflow indicators defined by Goble et al. [13] and operationalized by Wilkinson et al. [36], and against the FAIR4RS principles for the engine itself [6]. Table 12 summarizes compliance across four FAIR dimensions.

Principle	Indicator	OxyMake	Mechanism
Findable	F1: Unique ID	Native	Lockfile content hash
	F2: Rich metadata	Native	TOML is self-documenting
	F3: Searchable registry	Future	Workflow registry planned
Accessible	A1: Standard protocol	Native	Git, HTTP
	A2: Open format	Native	TOML, no vendor lock-in
Interoperable	I1: Standard serialization	Native	Parquet, CSV, columnar IPC
	I2: Workflow language	Partial	TOML + WDL/Snakemake bridge
	I3: Cross-platform	Native	Scaling ladder
Reusable	R1: Provenance	Native	Audit trail in state.db
	R2: Reproducibility	Native	Lockfile + content cache
	R3: Community standards	Native	Apache-2.0/MIT, open source

Table 12: FAIR compliance assessment. "Native" = built-in support; "Partial" = supported but not via a community standard format; "Future" = not yet implemented.

OxyMake achieves native compliance on 9 of 11 assessed indicators. The two gaps are: (1) no workflow registry for discovery (F3), which is designated future work; and (2) the TOML workflow format is not a community standard like CWL (I2), though the `ox translate` command provides bidirectional conversion with both Snakemake and WDL as bridges to the broader bioinformatics and genomics ecosystems.

6.10 Performance Summary

Benchmark	Result	Target	Status
DAG resolution @ 1K jobs	10 ms	$<$100 ms	Pass
DAG resolution @ 10K jobs	69 ms	$<$500 ms	Pass
DAG res. speedup @ 10K	33.3×	$>$1×	Pass
Startup time	7 ms	$<$200 ms	Pass
Binary size (default)	14.9 MB	$<$20 MB	Pass
Test suite	1,756 pass	—	Pass
Feature pass rate (CLI)	10/10	—	Pass

Table 13: Performance summary. All design targets are met with significant headroom.

7 Discussion

7.1 Positioning in the Landscape

OxyMake occupies a specific niche in the workflow tool landscape. Compared to Snakemake, it preserves the backward-chaining DAG paradigm while replacing heuristic change detection—

live mtimes in the Make tradition, working-tree-bound provenance records in Snakemake 7—with content-addressable caching (including transitive invalidation of deleted intermediates; Section 4.4), the Python DSL with declarative TOML, and the Python runtime with a Rust engine. Compared to the guix-cwl stack [29, 37], OxyMake targets the orchestration layers (L2–L4) of the FAIR ladder (Table 1) and leaves the binary substrate (L1) to a content-addressed package manager; whether the two contracts compose cleanly is a future direction (§7.3), not a claim made here. Compared to modern orchestrators (Dagster, Prefect, Temporal), OxyMake retains the simplicity of file-based rules while adding content-addressable caching and a machine-readable API. Compared to build systems (Bazel, Buck), OxyMake provides domain-specific features (wildcard expansion, environment management, gates) that build systems lack.

In the Mokhov et al. taxonomy [21, §3], OxyMake combines a *topological scheduler* (task order fixed before execution from the dependency graph) with *verifying traces* (skip a task when its recorded input/output hashes still match disk) augmented by content-addressing. All dependencies are resolved statically before execution, placing OxyMake in the same scheduler column as Make and Bazel rather than the suspending column occupied by Shake. Unlike Make, OxyMake replaces dirty-bit rebuilding (re-run whenever any input timestamp changes) with content-addressed verifying traces, and unlike Bazel, it targets scientific workflows with declarative TOML rules, wildcard expansion, and environment management.

7.2 Limitations and Future Work

TOML expressiveness. The decision to use TOML rather than a Turing-complete DSL limits the expressiveness of workflow definitions. Complex configuration generation must happen outside the Oxymakefile via scripts. While this preserves static parseability, it may frustrate users accustomed to Snakemake's Python flexibility. We note that the choice is not strictly binary: languages such as Starlark [4] (deterministic, sandboxed Python subset used by Bazel), CUE [34] (constraint-based configuration with types and validation), and Dhall [14] (a total functional language for configuration) occupy a middle ground between inert data formats and general-purpose languages, offering controlled expressiveness with static analysis guarantees. OxyMake opts for the simplest end of this spectrum—plain TOML—because workflow *specification* rarely needs computation; when it does, a config generation pattern (`python gen_config.py > config.toml`) or a minimal expression language (pure functions, no loops) provides an escape hatch without compromising the static parseability of the Oxymakefile itself.

No sandbox: undeclared inputs are trusted. As stated in the threat model (§3.1), OxyMake does not isolate rule execution, so the cache key's completeness is only as good as the user's input declaration: an undeclared read is a silent stale-reuse hazard. An opt-in sandboxed execution mode (namespace or seatbelt isolation that turns an undeclared read into a loud failure, recovering Nix's enforcement property) is future work; until it lands, the mitigation is declaration discipline plus the auditable `ox.lock` record.

SQLite on network filesystems. SQLite WAL mode does not work on NFS, Lustre, or GPFS—precisely the filesystems used on HPC clusters where SLURM runs. OxyMake requires `.oxymake/` to reside on local disk; the scheduler runs on the submission node, and compute nodes never touch SQLite. Multi-node coordination (multiple submission nodes) requires a future coordination layer. This local-disk requirement is also the discharge of the

`StateDbAtomicCommit` axiom that the model-checked multi-session guarantee rests on (§3.5).

Cache-key residual exclusions. Two ingredients are deliberately excluded from the cache key (Eq. 1). First, `call`-mode function bodies: the key covers the function *reference* (module path and name), but the module's source file is content-tracked only when declared as an input. Editing an imported module without redeclaring it can therefore serve a stale cached result; shell, inline-`run`, and script modes have no such gap (script file content is hashed into the key). Second, mutable container image tags: a `docker` or `apptainer` environment is hashed as the literal image reference, without resolving tags to digests—resolving would require a container-runtime round-trip per key computation and would break offline runs. A re-pushed `python:3.12-slim` therefore does not invalidate the cache; users who need this guarantee should pin images by digest (`python@sha256:...`), which the key then captures exactly.

In-memory passing limitations. The `call` mode's in-memory passing supports only types representable in the configured serialization format. Arbitrary Python objects (e.g., trained ML models) must be serialized to disk. A `pickle` codec provides a fallback but sacrifices cross-language compatibility.

Distributed executors. The local, SLURM, and Ray executors are fully implemented. The Kubernetes executor is designed (Section 4.3) but not yet implemented. The `Executor` trait and capability negotiation are in place; what remains is the backend-specific job submission, status polling, and log retrieval code for the Kubernetes backend.

Optimization passes. Of the six planned optimization passes (Section 4.1), only cache pruning is fully implemented. Task fusion, materialization elimination, group scheduling, critical path analysis, and partition planning are designed and have trait boundaries defined but await implementation. The current scheduler dispatches jobs in topological order without these optimizations; they will reduce execution time for `call`-mode workflows but do not affect correctness.

Maturity. OxyMake is in active development. The core engine (Table 5) implements the complete DAG pipeline from TOML parsing through parallel scheduling and execution, with all design targets met (Section 6). Production use should await the 1.0 release, after the API has stabilized through community feedback.

7.3 Future Work: Substrate Composition (Guix-CWL Direction)

The single largest future direction is the one this paper deliberately does *not* claim as a result: composing OxyMake's orchestration-level contract with a content-addressed binary substrate. OxyMake's cache key records an *output-equivalence* relation (§3.1)—do these outputs match what they would be if the inputs changed in a structured way?—while a content-addressed substrate such as the Guix store [7, 11] records an *input-equivalence* relation: does this input hash produce this binary? The conjecture (*OX-8*) is that the two witnesses are *orthogonal*—each checkable independently, neither subsuming the other—so that their conjunction is a strictly stronger reproducibility statement than either alone, of the kind the guix-cwl reference workflows [29] illustrate at the substrate layer. This is a design hypothesis, not an attested property: OX-8 has *no governing invariant* and is reported here as future work, not as a result.

Establishing it would require two pieces neither of which is on this project's roadmap. First, an optional `ox-exec-guix` execution crate (it does not exist; the design is sketched in `docs/design/ox-exec-guix-capability.md`) that runs rule bodies inside a `guix shell` environment. Second, an empirical *R0* attestation: a 2×2 matrix toggling the workflow input (axis: output equivalence) and the Guix manifest (axis: substrate equivalence) independently, with orthogonality holding iff the cache verdict varies only with the input axis and the store-path verdict only with the substrate axis. The harness is committed (`bench/orthogonality-r0/`) and the attestation template is at `docs/attestations/ox8-r0.md`, but execution needs a Linux host with both `ox` and `guix` installed—which the Darwin development box cannot provide—so no attestation is reported. A follow-on *R1* would extend the same matrix to `call`-mode rules (§4.2), whose in-process passing introduces an information channel not yet characterised against substrate equivalence.

Rolling a substrate into the engine is explicitly *not* the intended path. Each layer of the reproducibility ladder (Table 1) has a distinct owner in the broader ecosystem—Guix-HPC for L1, build-systems literature for L3, the RO-Crate community [31] for L4—and an engine that collapses all four into one binary inherits all four maintenance burdens. The `ox-exec-guix` crate above and a CWL reader/writer for `ox translate` (which today ships Snakemake and WDL bridges only) are therefore framed as invitations to a community-driven open-source effort rather than commitments of this paper. The substrate-composition story is a bounded future direction; the orchestration engine's contributions stand on their own.

7.4 A Note on Polyglot Layering

OxyMake assigns each layer a language chosen for the layer's constraints, not for uniformity. The orchestration core is Rust: the type system encodes pipeline-state invariants (`Rule` vs `ConcreteJob`) at compile time, and `Send + Sync` bounds make the concurrent scheduler thread-safe by construction. The workflow specification is TOML: deliberately not Turing-complete, statically parseable in microseconds, and analysable without execution—a property Snakemake's Python DSL cannot offer. Rule bodies execute in Python, R, Julia, or shell, selected per rule for the domain libraries each ecosystem encodes.

This split is not accidental polyglottism but a deliberate design choice: each subsystem uses the language whose properties match its function, with serialisation contracts (structured IPC, file I/O) at the boundaries. The cost is an extra language frontier per layer; the benefit is that no single language has to compromise. We make no broader claim here about how languages should be chosen in general; that discussion belongs in a separate venue.

8 Conclusion

We have presented OxyMake, a workflow orchestration engine that combines Snakemake's proven backward-chaining DAG paradigm with modern engineering in Rust. By replacing timestamp-heuristic change detection with content-addressable caching, introducing a three-graph architecture with pluggable optimization passes, supporting polyglot execution with in-memory data passing, providing daemon-free execution with a model-checked cooperative-claim protocol in the state layer, and offering a machine-readable API with structured event streams, OxyMake addresses the evolving requirements of computational workflows in an era of polyglot data science and large-scale distributed computation.

The founding principle—"Rust provides the engine, the workflow provides the intent"—enforces a clean separation that yields deterministic behavior (same inputs = same caching de-

cision, always), performance (69 ms DAG resolution for 10K jobs, 33.3× faster than Snakemake on the same workload), and extensibility (five plugin axes for executors, storage, environments, reporters, and format codecs).

OxyMake is open-source software under the Apache-2.0/MIT dual license, available at https://oxymake.noogram.dev.

Acknowledgments

We thank the GNU Guix, CWL, and FAIR-workflows communities, whose prior work informed this engine's design.

A Reproducibility and Falsifiability Ledger

The formal-specification discipline introduced in §3.5 is itself a falsifiable claim. The artefacts that make it falsifiable are committed to the repository and listed here so that a reviewer can audit them without running the code.

Specifications committed. The OxyMake repository ships three TLA+ modules under spec/tla/. Every number in the table below is reproducible from the repository: spec/tla/run-tlc.sh pins the TLC version by sha256, runs each committed configuration, and archives the full output under spec/tla/runs/ (committed); the figures here are read off those reference outputs.

Module	Lines	States	Depth	Invariants defended
CacheConsistency.tla	201	7 436	16	OX-1, OX-6 (stationary cache safety)
CooperativeClaim.tla	254	1 663 056	23	INV-2 (claim atomicity, stale-session reclaim, zombie terminal writes)
CancelPropagation.tla	168	32 768	22	INV-3 (CancelledNeverCached, JobFailedImpliesNoIntent)

Table 14: TLA+ specifications committed to spec/tla/. States = distinct states explored by TLC at the committed bounds; Depth = BFS depth of the complete state graph (both from spec/tla/runs/*.out, regenerated by run-tlc.sh). Each spec carries a TLC configuration file (.cfg) and opens with a preamble naming the invariant defended, the CSTAFP class, and the out-of-model substrate axioms inherited from docs/architecture/boundary.md. Two *red* configurations are committed alongside (CooperativeClaimUnguarded.cfg, CacheConsistencyNondetKey.cfg): each models a named defect (the pre-fix terminal UPDATE without session filter; an undeclared input leaking into the cache key) and is expected to *fail* — the proof that its green twin checks something falsifiable. run-tlc.sh -red runs them and errors if either passes.

Ledger files. Three append-only Markdown ledgers under spec/tla/ carry the sunset evidence:

- `spec/tla/TRACES.md` — one entry per TLC counterexample, with the violated invariant, TLC depth, root cause (Rust file and function), fix commit, and the pre-existing integration test that did not catch the trace.
- `spec/tla/REVIEWS.md` — one entry per six-month sunset review, recording the outcome (`kept`, `conditional`, `deleted`).
- `spec/tla/DESIGN-CHANGES.md` — one entry per design change motivated by a spec, even when no TLC trace was produced (the spec acted as a thinking tool).

As of this writing the ledgers carry their first non-bootstrap entries: `TRACE-001` (the zombie terminal-write class — a session whose claim was reclaimed could still terminalize the job; trace archived, reproducible via `run-tlc.sh -red`) and `DC-001` (the session-filter arm of the terminal `UPDATE`, sharpened by formalising the `Terminalize` action). Both cite the fixing Rust commit. The trace's chronology note records honestly that code review found the bug first and the revised spec reproduced it — the model as previously written could not express the bug class at all.

Review calendar. Six review dates are inscribed by ADR-015:

- **2026-06-15** — pilot review for `CacheConsistency.tla`: the spec must land at ≤ 80 lines with TLC depth 12 and a one-page note, operator validation under one hour. Failure reduces the ADR's ship-now scope.
- **2026-09-01** — intermediate health check.
- **2026-12-01** — *first sunset review.* Each spec must show ≥ 1 entry in `TRACES.md` *or* ≥ 1 entry in `DESIGN-CHANGES.md` within the prior six months. Otherwise the spec is deleted with a sunset citation.
- **2027-06-01, 2027-12-01** — successive six-month reviews. Three consecutive `conditional` outcomes mandate deletion.
- **2028-05-01** — long-horizon meta-review. Count production bugs whose root cause matches a named invariant; if zero, a meta-ADR reviews the entire `spec/tla/` directory.

Sunset rule. A specification that cannot point to either a TLC-produced trace violating a named invariant or a design change motivated by the spec within its six-month review window is deleted. A spec that no one reads, no one runs, and no one cites is worse than no spec at all: it implies a discipline without evidence. The ledger files make the discipline answerable to evidence rather than to authority.

References

[1] Mainak Adhikari, Tarachand Amgoth, and Satish Narayana Srirama. A survey on scheduling strategies for workflows in cloud environment and emerging trends. *ACM Computing Surveys*, 52(4):1–36, 2019.

[2] Enis Afgan, Dannon Baker, Bérénice Batut, Marius van den Beek, Dave Bouvier, Martin Čech, John Chilton, Dave Clements, Nate Coraor, Björn A. Grüning, et al. The Galaxy platform for accessible, reproducible and collaborative biomedical analyses: 2018 update. *Nucleic Acids Research*, 46(W1):W537–W544, 2018.

[3] Argoproj contributors. Argo Workflows: The workflow engine for Kubernetes. `https://github.com/argoproj/argo-workflows`, 2018. CNCF graduated project. Accessed: 2026-04-01.

[4] Bazel contributors. Starlark language specification. `https://github.com/bazelbuild/starlark/blob/master/spec.md`, 2018. Configuration language for Bazel, formerly Skylark. Accessed: 2026-04-01.

[5] Maxime Beauchemin and Apache Software Foundation. Apache Airflow: A platform to programmatically author, schedule, and monitor workflows. `https://airflow.apache.org`, 2015. Originally developed at Airbnb, Apache top-level project 2019. Accessed: 2026-04-01.

[6] Neil P. Chue Hong, Daniel S. Katz, Michelle Barker, Anna-Lena Lamprecht, Carlos Martinez, Fotis E. Psomopoulos, Jen Harrow, Leyla Jael Castro, Morane Gruenpeter, Paula Andrea Martinez, and Tom Honeyman. FAIR Principles for Research Software (FAIR4RS Principles). Recommendation, Research Data Alliance, 2022. Extends the FAIR Data Principles (Wilkinson 2016) to research software, adding software-specific indicators: machine-readable metadata, versioning, intelligible code, identifiable dependencies.

[7] Ludovic Courtès and Ricardo Wurmus. Reproducible and user-controlled software environments in HPC with Guix. In *Euro-Par 2015: Parallel Processing Workshops*, pages 579–591. Springer, 2015. GNU Guix as a from-source, content-addressed package manager for scientific computing. Establishes Guix's bit-reproducibility guarantee from input hash to binary output.

[8] Michael R. Crusoe, Sanne Abeln, Alexandru Iosup, Peter Amstutz, John Chilton, Nebojša Tijanić, Hervé Ménager, Stian Soiland-Reyes, Bogdan Gavrilović, Carole Goble, and The CWL Community. Methods included: Standardizing computational reuse and portability with the Common Workflow Language. *Communications of the ACM*, 65(6):54–63, 2022.

[9] Ewa Deelman, Karan Vahi, Gideon Juve, Mats Rynge, Scott Callaghan, Philip J. Maechling, Rajiv Mayani, Weiwei Chen, Rafael Ferreira da Silva, Miron Livny, and Kent Wenger. Pegasus, a workflow management system for science automation. *Future Generation Computer Systems*, 46:17–35, 2015.

[10] Paolo Di Tommaso, Maria Chatzou, Evan W. Floden, Pablo Prieto Barja, Emilio Palumbo, and Cedric Notredame. Nextflow enables reproducible computational workflows. *Nature Biotechnology*, 35(4):316–319, 2017.

[11] Eelco Dolstra. *The Purely Functional Software Deployment Model.* PhD thesis, Utrecht University, 2006.

[12] Stuart I. Feldman. Make – a program for maintaining computer programs. *Software: Practice and Experience*, 9(4):255–265, 1979.

[13] Carole Goble, Sarah Cohen-Boulakia, Stian Soiland-Reyes, Daniel Garijo, Yolanda Gil, Michael R. Crusoe, Kristian Peters, and Daniel Schober. FAIR Computational Workflows. *Data Intelligence*, 2(1-2):108–121, 2020.

[14] Gabriel Gonzalez. Dhall: A programmable configuration language, 2024. A total functional language that guarantees termination.

[15] Google. Bazel: A fast, scalable, multi-language and extensible build system. `https://bazel.build`, 2015. Open-source release of Google's internal Blaze build system. Accessed: 2026-04-01.

[16] Johannes Köster and Sven Rahmann. Snakemake—a scalable bioinformatics workflow engine. *Bioinformatics*, 28(19):2520–2522, 2012.

[17] Leslie Lamport. *Specifying Systems: The TLA+ Language and Tools for Hardware and Software Engineers*. Addison-Wesley, 2002.

[18] Meta Engineering. Build faster with Buck2: Our open source build system. `https://engineering.fb.com/2023/04/06/open-source/buck2-open-source-large-scale-build-system/`, 2023. Meta Engineering Blog. Accessed: 2026-04-01.

[19] Neil Mitchell. Shake before building: Replacing Make with Haskell. In *Proceedings of the 17th ACM SIGPLAN International Conference on Functional Programming*, pages 55–66, 2012.

[20] Andrey Mokhov, Neil Mitchell, and Simon Peyton Jones. Build Systems à la Carte. In *Proceedings of the ACM on Programming Languages*, volume 2, pages 1–29, 2018.

[21] Andrey Mokhov, Neil Mitchell, and Simon Peyton Jones. Build systems à la carte: Theory and practice. *Journal of Functional Programming*, 30:e11, 2020.

[22] Felix Mölder, Kim Philipp Jablonski, Brice Letcher, Michael B. Hall, Christopher H. Tomkins-Tinch, Vanessa Sochat, Jan Forster, Soohyun Lee, Sven O. Twardziok, Alexander Kanitz, Andreas Wilm, Manuel Holtgrewe, Sven Rahmann, Sven Nahnsen, and Johannes Köster. Sustainable data analysis with Snakemake. *F1000Research*, 10:33, 2021.

[23] Philipp Moritz, Robert Nishihara, Stephanie Wang, Alexey Tumanov, Richard Liaw, Eric Liang, Melih Elibol, Zongheng Yang, William Paul, Michael I. Jordan, and Ion Stoica. Ray: A distributed framework for emerging AI applications. In *Proceedings of the 13th USENIX Symposium on Operating Systems Design and Implementation (OSDI 18)*, pages 561–577, 2018.

[24] Chris Newcombe, Tim Rath, Fan Zhang, Bogdan Munteanu, Marc Brooker, and Michael Deardeuff. How Amazon Web Services uses formal methods. *Communications of the ACM*, 58(4):66–73, 2015.

[25] Jack O'Connor, Jean-Philippe Aumasson, Samuel Neves, and Zooko Wilcox-O'Hearn. BLAKE3: One function, fast everywhere. `https://github.com/BLAKE3-team/BLAKE3`, 2020. Accessed: 2026-03-24.

[26] Jack O'Connor, Jean-Philippe Aumasson, Samuel Neves, and Zooko Wilcox-O'Hearn. BLAKE3 specification. https://github.com/BLAKE3-team/BLAKE3-specs/blob/master/blake3.pdf, 2020. Benchmarks: 6.9 GiB/s single-threaded on Cascade Lake-SP 8275CL (16 KiB input); AVX-512 exceeds 8 GB/s. Accessed: 2026-04-01.

[27] Roger D. Peng. Reproducible research in computational science. *Science*, 334(6060):1226–1227, 2011.

[28] petgraph contributors. petgraph: Graph data structure library for Rust. https://github.com/petgraph/petgraph, 2024. $O(|V| + |E|)$ topological sort via Kahn's algorithm. Accessed: 2026-03-24.

[29] Pjotr Prins. CWL-workflows: Guix-managed Common Workflow Language reference workflows. Software repository, https://github.com/pjotrp/CWL-workflows, 2020. Software artifact (fork of hacchy1983/CWL-workflows). Reference workflows illustrating how a CWL runner composes with Guix-managed environments: orchestration (CWL/cwltool) and substrate (Guix store) are deliberately decoupled. README dated 2020. Accessed: 2026-05-28.

[30] Matthew Rocklin. Dask: Parallel computation with blocked algorithms and task scheduling. In *Proceedings of the 14th Python in Science Conference (SciPy 2015)*, pages 126–132, 2015.

[31] Stian Soiland-Reyes, Peter Sefton, Mercè Crosas, Leyla Jael Castro, Frederik Coppens, José M. Fernández, Daniel Garijo, Björn Grüning, Lorraine La Rosa, Stefano Leo, Eoghan Ó Carragáin, Marc Portier, Ana Trisovic, RO-Crate Community, Paul Groth, and Carole Goble. Packaging research artefacts with RO-Crate. *Data Science*, 5(2):97–138, 2022. Specifies RO-Crate, a JSON-LD packaging format that bundles a workflow with its inputs, outputs, and provenance into a single FAIR-compliant artefact. Adopted by the WorkflowHub registry.

[32] Victoria Stodden, Marcia McNutt, David H. Bailey, Ewa Deelman, Yolanda Gil, Brooks Hanson, Michael A. Heroux, John P. A. Ioannidis, and Michela Taufer. Enhancing reproducibility for computational methods. *Science*, 354(6317):1240–1241, 2016.

[33] Haluk Topcuoglu, Salim Hariri, and Min-You Wu. Performance-effective and low-complexity task scheduling for heterogeneous computing. *IEEE Transactions on Parallel and Distributed Systems*, 13(3):260–274, 2002.

[34] Marcel van Lohuizen. CUE: Validate, define, and use dynamic and text-based data, 2024. A constraint-based configuration language with types and validation.

[35] Kate Voss, Geraldine Van der Auwera, and Jeff Gentry. Full-stack genomics pipelining with GATK4 + WDL + Cromwell. F1000Research 6(ISCB Comm J):1381, 2017.

[36] Sean R. Wilkinson, Meznah Aloqalaa, Khalid Belhajjame, Michael R. Crusoe, Bruno de Paula Kinoshita, Luiz Gadelha, Daniel Garijo, Stian Soiland-Reyes, et al. Applying the FAIR Principles to computational workflows. *Scientific Data*, 12:328, 2025.

[37] Ricardo Wurmus, Bora Uyar, Brendan Osberg, Vedran Franke, Alexander Gosdschan, Katarzyna Wreczycka, Jonathan Ronen, and Altuna Akalin. PiGx: Reproducible genomics analysis pipelines with GNU Guix. *GigaScience*, 7(12):giy123, 2018. Demonstrates Guix as

the substrate layer for bioinformatics pipeline reproducibility; orchestration layer remains Snakemake-like.